

A Gentle Introduction to Assembly Language Programming

This textbook provides a gentle introduction to assembly language programming. What makes this introduction “gentle” is that it assumes the reader is already comfortable with C or C++ coding. We use this assumed knowledge to **bridge** backward towards the low level ISA (Instruction Set Architecture).

We drive home a very sharp point:

Assembly language is nothing to be scared of!

[Quick Link to Table of Contents](#)

Front Matter

For Whom Is This Book Intended?

As mentioned, if you are already familiar with C (or languages descended from C such as C++), this book begins with what you already know.

Later chapters dive more deeply into the corners and recesses of the ARM V8 ISA and are suitable for those wishing to master the rich instruction set of the 64 bit ARM processors.

Can This Book Be Used In Courses Covering Assembly Language?

Yes, absolutely.

Calling Convention Used In This Book

Assembly language programming is quite closely dependent upon the underlying hardware architecture. The host operating environment plays an outsized role in determining how assembly language programs are constructed. A “calling convention” refers to how functions are called and how parameters are passed.

Originally, this book taught only the ARM LINUX conventions. However, over time, we developed a suite of macros that make it much easier to write code for use either on MacOS or on LINUX.

This link will lead to a current copy of them as well as documentation. Macros that make programming a bit easier are also included.

This chapter provides some additional information about Apple Silicon assembly language programming.

You’ll notice that we make use of the C-runtime directly rather than make OS system calls. So, for instance, if we want to call `write()`, we call `write` from the assembly language.

This version of the system call `write` is a wrapper function built into the C-runtime (CRT) which handles the lower level details of performing a system call. See this chapter for what actually happens inside these wrapper functions.

The benefit of using the CRT wrappers is that there are differences between the distributions and architectures that are masked by using the CRT wrappers. Therefore, when you use the wrappers rather than the direct method of making system calls, your code will be more portable.

A Lot of Names

As commendable as the ARM designs are, ARM's naming conventions for their Intellectual Properties are horrid. In this book, AARCH64 and ARM V8 are taken to be synonyms for the 64 bit ARM Instruction Set Architecture (ISA).

It is very difficult to find documentation at the ARM site because they have *so many versions*, so many names for the same thing and so much documentation in general. It really can be maddening.

Within the text we will provide germane links as appropriate.

Here is a link to “a” main instruction set page.

What you need to work with assembly language on Linux

Getting the tools for assembly language development is quite straightforward - perhaps you already have them. Using `apt` from the Linux terminal, say:

```
sudo apt update
sudo apt install build-essential gdb
```

On the Macintosh type:

```
xcode-select --install
```

into a terminal and follow directions. Note that `gdb` is replaced by `lldb` with just enough differences to make you cry.

Then you'll need your favorite editor. We ourselves use `vi` for quick edits and Visual Studio Code for any heavy lifting.

How to build an assembly language program

We use `gcc`, the C “compiler”. `g++` could also be used. On the Mac, `clang` can also be used.

What sense does that make... using the “compiler” to “compile” assembly language?

Well, to answer that one must understand that the word “compiler” refers to only one step in a build sequence. What we talk about as being the “compiler” is actually an umbrella that includes:

- A preprocessor that acts on any `#` preprocessor command like `#include`. These commands are not part of C or C++. Rather they are commands to the preprocessor.

Note that `gcc` will invoke the C preprocessor if your assembly language file ends in `.S` - capital S. It may or may not be invoked if your file ends in a lower case `s` or any other file extension depending upon your system.

- The *actual* compiler, whose job it is to turn high level languages such as C and C++ into assembly language.
- The assembler, which turns assembly language into machine code which is not quite ready for execution.
- And finally, the linker, which combines potentially many intermediate machine code files (called object files) with potentially many library files (statically linked `.a` files on Linux or `.lib` files on Windows; dynamically linked `.so` files on Linux or `.dll` files on Windows). The linker is the last step in this chain.

Here is a video explaining this process.

We use `gcc` and `g++` directly because, being umbrellas, they automate the above steps and automatically link with the CRT.

Suppose you've implemented `main()` in a C file (`main.c`) and want to make use of an assembly language file you have written (`asm.S`). It can be done in several ways.

All at once

```
gcc main.c asm.S
```

That's all you need for a minimal build. The resulting program will be written to `a.out`. All the intermediate files that are generated will be removed.

Modularly

```
gcc -c main.c
gcc -c asm.S
gcc main.o asm.o
```

Used in this way, `.o` files are left on disk. Using the previous method, the `.o` files are removed without you seeing them.

If there are no C or C++ modules used

Suppose `main()` is implemented in assembly language and `main.S` is self-contained, then simply:

```
gcc main.S
```

Often, you will want to enable the debugger `gdb` or `lldb`. Do this:

```
gcc -g main.S
```

Without the `-g` command line option, your debugger may not properly operate.

The C Preprocessor To repeat, if you want `gcc` to run your code through the C preprocessor (for handling `#include` for example), name your assembly language source code files with a capital S. So, on Linux:

```
gcc main.s
```

Will not go through the C preprocessor but

```
gcc main.S
```

will.

Programs called by the “Compiler” To drive home the point that the “compiler” is an umbrella, using `gcc` to “compile” a program causes the following to be called on Ubuntu running on ARM:

```
/usr/bin/cpp
/usr/lib/gcc/aarch64-linux-gnu/11/cc1
/usr/bin/as
/usr/lib/gcc/aarch64-linux-gnu/11/collect2 which is...
/usr/bin/ld
```

`cpp` is the C preprocessor - it is a general tool used by other languages as well (C++, for example).

`cc1` is the actual compiler.

`as` is the assembler.

`ld` is the linker.

You can see why we default to using the umbrella command in this book.

Table of Contents

Section 1 - Bridging from C / C++ to Assembly Language

We start by providing what we’re calling “bridging” from C and C++ to assembly language. We use the knowledge you already have to learn new knowledge - how cool is that!

Chapter	Markdown	PDF
0	Kickstart	Link
1	Hello World	Link
2	If Statements	Link

Chapter	Markdown	PDF
3	Loops	
3a	While Loops	Link
3b	For Loops	Link
3c	Implementing Continue	Link
3d	Implementing Break	Link
4	Interludes	
4a	Registers	Link
4b	Load and Store	Link
4c	More About <code>ldr</code>	Link
4d	Register Sizes	Link
4e	Hexadecimal	Link
5	<code>switch</code>	Link
6	Functions	
6a	Calling and Returning	Link
6b	Passing Parameters	Link
6c	Example of calling some common C runtime functions	Link
7	FizzBuzz - a Complete Program	Link
8	Structs	
8a	Alignment	Link
8b	Defining	Link
8c	Using	Link
8d	What is “this”	Link
9	<code>const</code>	Link

Section 2 - Floating Point

Floating point operations use their own instructions and their own set of registers. Therefore, floating point operations are covered in their own section:

Chapter	Markdown	PDF
0	Chapter Overview	Link
1	What Are Floating Point Numbers?	Link
2	Registers	Link
3	Truncation and Rounding	Link
4	Literals	Link
5	<code>fmov</code>	Link
6	Half Precision Floats	Link
7	NEON SIMD Not Yet Written	Link

Section 3 - Bit Manipulation

What would a book about assembly language be without bit bashing?

Chapter	Markdown	PDF
1	Bit Fields	
1a	Without Bit Fields	Link
1b	With Bit Fields	Link
1c	Review of Newly Described Instructions	Link
2	Endianness	Link

Section 4 - More Stuff

In this section, we present miscellaneous material including our “world famous lecture” on debugging. This lecture has been invited at several colleges and universities. It is intended for audiences working with languages like C, C++ and assembly language but some of the lessons contained therein are applicable to all languages.

Chapter	Markdown	PDF
1	Apple Silicon	Link
2	Apple / Linux Convergence	Link
3	Variadic Functions	Link
4	Under the hood: System Calls	Link
5	Determining string literal lengths for C functions	Link
6	Calling Assembly Language From Python	Link
7	Atomic Operations	Link
8	Jump Tables	Link
9	argv	ASM CODE
10	spin-locks	Link
-	Debugging Lecture	PPTX

Macro Suite

As indicated immediately above, the macro suite can be found [here](#).

Projects

Here are some project specifications to offer a challenge to your growing mastery. Here are very brief descriptions presented in alphabetical order.

- Perhaps before you tackle these, check out the fully described FIZZBUZZ program first.
- Then try this as your very first project. With some blank lines and comments it weighs in at 35 lines.
- The DIRENT project demonstrates how a complex **struct** can be used in assembly language.
- The PI project demonstrates floating point instructions. The program will “throw darts at a target,” calculating an approximation of PI by tracking how many darts “hit the target” versus the total number of darts “thrown”.
- The SINE project stresses floating point math and functions.
- The SNOW project uses 1970’s era tech to animate a simple particle system. This project demonstrates a reasonable design process of breaking down complex problems into simpler parts.
- The WALKIES presents a cute little animation demonstrating looping with some pointer dereferencing.

About The Author

Perry Kivolowitz’s career in the Computer Sciences spans just under five decades. He launched more than 5 companies, mostly relating to hardware, image processing and visual effects (for motion pictures and television). Perry received Emmy recognition for his work on the *The Gathering*, the pilot episode of *Babylon 5*. Later he received an Emmy Award for Engineering along with his colleagues at SilhouetteFX, LLC. SilhouetteFX is used in almost every significant motion picture for rotoscoping, paint, tracking, 2D to 3D reconstruction, compositing and more.

In 1996 Perry received an Academy Award for Scientific and Technical Achievement for his invention of Shape Driven Warping and Morphing. This is the technique responsible for many of the famous effects in *Forrest Gump*, *Titanic* and *Stargate*.

Perry retired from college teaching after nineteen years — ten at UW Madison and nine at Carthage College.

Perry’s early career ran deep in Unix and the Amiga. As an undergraduate he read the V6 Unix kernel from source on a PDP-11/60; he later taught Unix internals at Bell Labs. He wrote what is believed to be the first software keylogger. In 1985 his work appeared on *Fred Fish Disk #1* — the first volume of the longest-running public-domain software library in personal-computing history. His commercial work through **ASDG**, **TAD**, **ADPro**, and **Elastic Reality** centered on a “Loaders → Operators → Savers” pipeline architecture he invented and carried through five product generations across desktop publishing, multimedia, video, and film. The same architecture has most recently evolved into

a **Sensor** $\rightarrow$ **Operator** $\rightarrow$ **Emitter** (SOE) pipeline in his current open-source project **GlowUp** (github.com/pkivolowitz/lifx) — a generalized sensor-fusion and effectuation platform that began as smart-lighting control and now runs an ever-growing share of his home.

Assembly language is a passion for Perry having worked in the following ISAs (in chronological order):

- Univac 1100
- Digital Equipment Corporation PDP-11
- Digital Equipment Corporation VAX-11
- Motorola 68000
- ARM beginning with AARCH64

This work is dedicated to my wife Sara and sons Ian and Evan.

A Note on Authorship and AI Assistance

As of 19 April 2026, Perry has begun using Claude (Anthropic’s AI assistant) as a writing and editing aid on this book. All material preceding that date was produced by the author unaided. Ongoing AI-assisted changes are identifiable in the git history by commit date and by the supporting tooling introduced in `scripts/` and `.github/` around that time.

Gratuitous Plugs

Perry has created a library of about 200 programming projects suitable for CS 1, CS 2, Data Structures, Networking, Operating Systems and Computer Organization classes. If a publisher of CS text books (or other CS related content) would be interested in purchasing the library, please reach out.

Also, check out *Get Off My L@wn*, a Zombie novel for coders.

You read that right... elite programmer Doug Handsman retires to his wife Ruth Ann’s native northern Wisconsin. And then, well, the apocalypse happens. Bummer.

Rated 4.3 out of 5 with more than 70 reviews, it’s a fun read and costs next to nothing.